

Base de Datos II

SQL PROCEDURAL
Triggers, Stored Procedures



SQL Procedural

Posibilita el uso de código procedural conjuntamente con sentencias SQL que son almacenadas dentro de la BD. El código procedural es ejecutado por el DBMS cuando es invocado (directa o indirectamente) por el usuario de la BD.

Ventajas:

- *Aísla partes comunes existentes en las aplicaciones, delegándolas en el DBMS.*
- *Eficiencia (en general, cerca de un orden de magnitud)*

Desventajas:

- *Cada proveedor de BD tiene su propio lenguaje procedural.*

El SQL-1999 incorporó estas características, pero poco de lo definido anteriormente se ajustaba a un estándar.

SQL Procedural

Se puede utilizar SQL Procedural para definir:

- ✓ *Trigger: Es un procedimiento que es invocado automáticamente por el DBMS en respuesta a un evento específico de la BD.*
- ✓ *Stored Procedure: Es un procedimiento que es invocado explícitamente por el usuario.*
- ✓ *Función: puede ser predefinida o definida por el usuario para realizar operaciones específicas sobre los datos, y pueden ser invocadas desde un trigger, stored procedure o explícitamente.*

Extensión en SQL-1999

SQL-1999 ha extendido el SQL-1992 en varios aspectos, uno de ellos el procedural.

Extensiones Procedimentales:

Sentencias compuestas (agruparlas en bloques begin end)

Declaración de variables y constantes.

Sentencias de Flujo de control (consultar manual del DBMS)

- ✓ If-then-else (elsif)

- ✓ While

- ✓ For (itera sobre los elementos de una tabla resultado)

- ✓ Loop y repeat

Usos de cada tipo de SQL Procedural

Algunos usos típicos de cada elemento.

Triggers:

- Actualización de valores calculados del negocio. Por ejemplo Saldo de una cuenta, total de una factura, etc.
- Control de reglas del negocio no posibles de hacer en SQL Declarativo.

Procedimientos y Funciones:

- Todo proceso que necesite de control de flujo, bucles, etc. (if, for, while, cursores, etc.)
 - Reportes.
 - Cálculos intensivos.
 - Abstracción de tablas (para trabajo en capas)

Procedimientos/Funciones en Postgres

Para Postgres todos son funciones, sólo que hay funciones que devuelven void (Procedimientos).

```
CREATE [ OR REPLACE ] FUNCTION nombre_funcion([ [ argmodo ] [
argnombre ] argtipo [, ...] ])
RETURNS tipo AS $$
[ DECLARE ] [ declaraciones de variables ]
BEGIN
    codigo
END;
$$ LANGUAGE plpgsql ;
```

Procedimientos/Funciones en Postgres

Ejemplos de Función.

```
CREATE OR REPLACE FUNCTION Sumador(integer)
RETURNS integer AS $$
BEGIN
    RETURN $1 + 1;
END; $$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION Sumador(unNumero integer)
RETURNS integer AS $$
BEGIN
    RETURN unNumero + 1;
END; $$ LANGUAGE plpgsql;
```

Select Sumador(19);

Procedimientos/Funciones en Postgres

Declaración de Variables

```
CREATE Function Ejemplo2(integer, integer) .....
```

```
DECLARE
```

```
    numero1 ALIAS FOR $1; // Primer parámetro
```

```
    numero2 ALIAS FOR $2; // Segundo parámetro
```

```
    constante CONSTANT integer := 100;
```

```
    resultado INTEGER;
```

```
    resultado_txt TEXT DEFAULT 'Texto por defecto';
```

```
    tipo_reg voluntario%rowtype; // variable del tipo registro
```

```
    tipo_col voluntario.nombre%type; // variable del tipo columna
```

```
.....
```

Procedimientos/Funciones en Postgres

CURSORES

Un cursor es un tipo de variable que nos permite acceder a las filas de un conjunto de datos (Tabla, consulta, etc.) en forma secuencial, no pudiendo volver a una fila anterior una vez que se avanza el puntero.

Todo el acceso a cursores en PL/pgSQL es a través de variables del tipo *cursor*, las cuales son siempre del tipo de datos especial *refcursor*.

Una forma de crear una variable tipo cursor es declararla como de tipo *refcursor* o usar la sintaxis de declaración de cursor, la cual en general es:

```
nombre CURSOR [ ( argumentos ) ] FOR select_query ;
```

Por ejemplo:

DECLARE

```
curs1 refcursor; (puede utilizarse para cualquier consulta)
```

```
curs2 CURSOR FOR SELECT * from voluntario; // solo se utiliza con la consulta declarada
```

```
curs3 CURSOR (key int) IS
```

```
    SELECT * from voluntario where id_voluntario = key; // consulta parametrizada, key será reemplazado por un valor de parámetro entero cuando se inicialice el cursor
```

Procedimientos/Funciones en Postgres

CURSORES

Para utilizar un cursor hay que abrirlo, para hacerlo depende del tipo de cursor.

- **GENERIC**
 - `OPEN CURSOR FOR SELECT .....`
 - `OPEN curs1 for select * from Pais;`
 - `OPEN CURSOS FOR EXECUTE ....`
 - `OPEN curs1 for execute "select * from Pais";`
- **Ya Especificado (BOUNDED CURSOR)**
 - `OPEN curs2;`
 - `OPEN curs3(4444);`
- Para traer fila a fila se utiliza el `FETCH` y no olvidarse de cerrarlo con un `CLOSE`.
 - `Fetch curs2 into variable;`
 - `Close curs2;`

Utilizar la variable `FOUND` para ver si trajo una fila o no.

Procedimientos/Funciones en Postgres

CURSORES

CREATE FUNCTION

DECLARE

 cursor cur1 for select * from pais;

 mifila pais%rowtype;

 mensaje TEXT DEFAULT 'no hay registros';

Begin

 open cur1;

 fetch cur1 into mifila;

 if FOUND then

 mensaje := 'Por lo menos hay un registro';

 end if;

 close cur1;

 Return mensaje;

end

.....

Procedimientos/Funciones en Postgres

Funciones que devuelven una TABLA

```
create function VoluntariosPorApellido(parte_apellido varchar)
returns
    TABLE(nro_voluntario numeric, apellido_nombre varchar)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT v.nro_voluntario,
               cast((v.apellido||', '||v.nombre) as varchar)
        FROM unc_esq_voluntario.voluntario v
        WHERE upper(v.apellido) LIKE ('%'||upper(parte_apellido)||'%');
END;
$$;
```

```
select *
from VoluntariosPorApellido('co');
```

Procedimientos/Funciones en Postgres

Funciones que devuelven una TABLA

```
CREATE FUNCTION voluntarioscadax(x integer) RETURNS
TABLE(nro_voluntario numeric, apellido varchar, nombre varchar)
AS $$
DECLARE
    var_r record;
    i int;
BEGIN
    i := 0;
    FOR var_r IN (
        SELECT v.nro_voluntario, v.apellido, v.nombre
        FROM unc_esq_voluntario.voluntario v)
    LOOP
        IF (i % x = 0) THEN
            nro_voluntario := var_r.nro_voluntario;
            apellido := var_r.apellido;
            nombre := var_r.nombre;
            i := 0;
            RETURN NEXT;
        END IF;
        i := i + 1;
    END LOOP;
END;
$$; LANGUAGE plpgsql;
```

```
select *
from voluntarioscadax(3);
```

Restricciones.....

Se puede pensar en cuatro tipos de restricciones

1. A nivel de Atributo

El Sueldo no puede ser negativo

La fecha de Nacimiento tiene que estar en este siglo

Esto por supuesto que puede ser controlado con triggers, pero lo más correcto es controlarlo con un constraint.

Alter table add constraint chk_sueldo check (sueldo >= 0);

Restricciones.....

2. A nivel de Fila

Involucra el control de una restricción entre columnas de una misma fila o tupla.

La fecha de Ingreso tiene que ser menor que la de egreso.

Si el sueldo es mayor que 5000 la comisión tiene que ser 0.

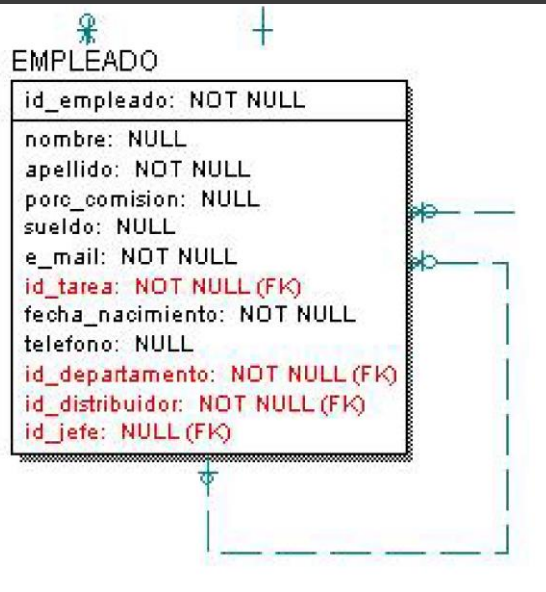
Esto por supuesto también puede ser controlado por Triggers, pero sigue siendo lo más correcto controlarlo con un constraint.

*Alter table add constraint chk_sueldo_comision check
((sueldo > 5000 and comision = 0) or (sueldo <= 5000));*

Restricciones.....

3. A nivel de Tabla

Involucra el control de una restricción de UNA sola tabla.



El promedio de edad de cada Departamento no puede ser mayor que 50 años.
Ningún empleado puede ganar más que su jefe.

Esto por supuesto también puede ser controlado por triggers, PERO SIGUE SIENDO lo más correcto controlarlo con un constraint.

```
Alter table EMPLEADO add constraint chk_sueldo_comision check not exists (  
  select 1  
  from empleado  
  group by id_distribuidor, id_departamento  
  having avg( months_between ( sysdate, fecha_nacimiento ) ) > (50 * 12) );
```

*Postgres **NO** implementa este tipo de checks,
Postgres no permite un **select** dentro de un
constraint.... ☹*

Entonces.....

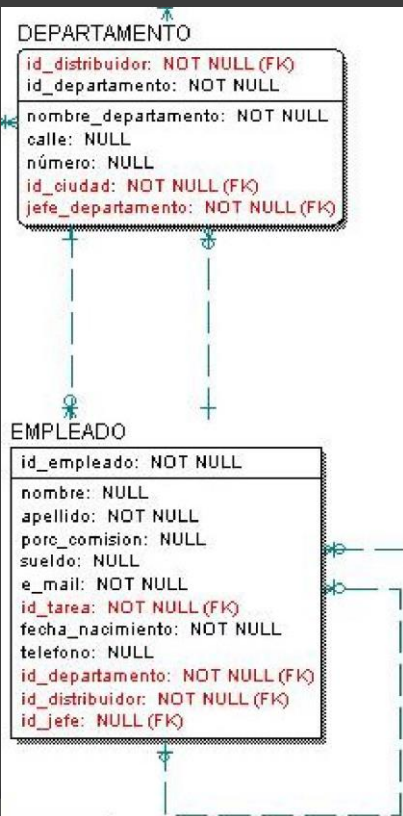
*Lamentablemente se tendrá que pensar.... se
deberá analizar ante qué eventos y en qué tiempo
se debe disparar él o los triggers asociados a
EMPLEADO*

Restricciones.....

4. Generales (ASSERTIONS)

Involucra el control de una restricción de MÁS DE UNA tabla.

Todos los empleados deben trabajar en el mismo Departamento que el jefe del Departamento.



*Ninguna base de datos comercial implementa Assertions.
Se deberá controlar la restricción con triggers en varias tablas.*

```
Create Assertion ast_empleado_jefe check not exists (  
  select 1  
  from empleado e  
    join departamento d on ( .... ) // empleado con departamento  
    join empleado j on ( .... ) // departamento con empleado (jefe)  
  where ( ( e.id_departamento <> j.id_departamento) or  
          (e.id_distribuidor <> j.id_distribuidor) )  
);
```

Se deben generar triggers en DEPARTAMENTO y en EMPLEADO para que controlen esta restricción, ya que si sólo se hiciera sólo en una, por ejemplo EMPLEADO, uno podría cambiar el jefe en la tabla de DEPARTAMENTO y el trigger en Empleado no se despertaría.

Claramente se deberá analizar la granularidad, eventos y tiempo de activación de cada trigger involucrado.



Preguntas ??

Links

<http://www.postgresqltutorial.com>

<http://www.postgresql.com>